

REPORTE TAREA 2

ALGORITMOS Y COMPLEJIDAD

«¿Quien ganará? Greedys vs Dinámico vs Bruto.»

Oscar Acosta

13 de junio de 2026

16:23

1. Entorno experimental

Para evaluar el rendimiento y la correctitud de los algoritmos implementados, los experimentos se ejecutaron en un entorno con las siguientes especificaciones de hardware, software y conjuntos de datos:

1.1. Entorno de Hardware y Software

Las pruebas experimentales se llevaron a cabo en un equipo con las siguientes características técnicas:

- **Procesador:** AMD Ryzen 5 4500U with Radeon Graphics (2.38 GHz).
- **Memoria RAM:** 16GB DDR4.
- **Sistema Operativo:** Windows 11 con el Subsistema de Windows para Linux (WSL) ejecutando la distribución de Ubuntu.
- **Compilador:** g++ (GCC) bajo el estándar C++17, con automatización de compilación y ejecución mediante directivas make.
- **Entorno de Análisis:** Entorno Python 3 con las librerías pandas y matplotlib para el procesamiento de las mediciones y la generación de gráficos.

1.2. Conjunto de Datos (Datasets)

Los casos de prueba fueron generados de manera sintética y automatizada mediante el script `testcases_generator.py`. Cada instancia generada respeta estrictamente las restricciones de dominio establecidas para el problema AniMaraton: los minutos máximos se encuentran en el rango $1 \leq$

$M \leq 3000$ y la energía en $1 \leq E \leq 500$. Adicionalmente, se aseguró que la cantidad de capítulos por serie no supere los 30 ($q_i \leq 30$), con un tope global máximo de 700 capítulos por archivo.

El *dataset* se dividió en tres categorías de evaluación:

- **Casos pequeños** ($N \in \{3, 5, 8\}$): Utilizados para validar la exactitud del algoritmo de Fuerza Bruta y confirmar que sus resultados coincidan con la solución óptima de la Programación Dinámica.
- **Casos medianos** ($N \in \{20, 40, 80\}$): Empleados para comparar la calidad de las soluciones heurísticas (Greedy) frente a la solución óptima.
- **Casos grandes** ($N \in \{100, 150, 200\}$): Destinados a pruebas de estrés para medir el rendimiento y escalabilidad de los algoritmos eficientes, escenarios en los cuales se asume empírica y teóricamente que la Fuerza Bruta se vuelve inejecutable en tiempos razonables.

1.3. Diseño y Análisis de Algoritmos

En esta sección se detalla el diseño de los cuatro enfoques implementados para resolver el problema de la AniMaratón, explicando la lógica de su funcionamiento y analizando sus complejidades teóricas de tiempo y espacio.

1.3.1. Fuerza Bruta (Backtracking)

Para la Fuerza Bruta se utilizó una estrategia de exploración exhaustiva mediante recursión (*Backtracking*) [1]. El problema se pensó como un árbol de decisión donde cada nivel representa un anime diferente. En cada nivel, el algoritmo tiene $q_i + 1$ opciones disponibles: elegir no ver ningún capítulo, o ver desde el primero hasta el capítulo k (con k entre 1 y q_i).

Para evitar recorrer ramas completamente innecesarias y acelerar el proceso, se programó una poda por factibilidad dentro del ciclo de llamadas. Como cada capítulo consume tiempo y energía positivos, en el momento en que la suma parcial de cualquiera de estos dos recursos supera el límite global de la mochila (M o E), el algoritmo rompe el ciclo de inmediato con un *break*. Esto evita seguir profundizando en soluciones que ya son inválidas para el sistema.

Complejidad Temporal Teórica: En el peor caso posible, donde todas las combinaciones generadas fueran válidas y no operara la poda por recursos, el algoritmo tendría que revisar absolutamente todas las opciones. Como para cada anime hay $q_i + 1$ decisiones posibles, el total de hojas a evaluar en el árbol sería el producto de las opciones de todos los animes:

$$T(N) = O\left(\prod_{i=1}^N (q_i + 1)\right) \quad (1)$$

Considerando las restricciones fijas de la tarea, donde cada anime puede tener como máximo 30 capítulos ($q_i \leq 30$), el número de opciones por nivel se reduce a una constante máxima de 31. Por lo tanto, la complejidad en el peor caso se comporta de forma puramente exponencial:

$$T(N) = O(31^N) \quad (2)$$

Esto explica numéricamente por qué este enfoque se vuelve inutilizable al aumentar ligeramente el número de series.

Complejidad Espacial Teórica: La memoria que consume este enfoque depende únicamente de la pila de llamadas del sistema (*call stack*) debido a la recursión. Como el algoritmo procesa un anime a la vez antes de seguir bajando en la rama del árbol, la profundidad máxima de la pila es igual a la cantidad total de animes N . En consecuencia, la complejidad espacial teórica es lineal:

$$S(N) = O(N) \quad (3)$$

Esto calza perfectamente con los resultados de las mediciones, donde el uso de memoria de la Fuerza Bruta se mantuvo bajísimo y plano a lo largo de su ejecución.

1.3.2. Greedy

Los algoritmos Greedy se diseñaron transformando el problema de los prefijos consecutivos en una lista de opciones independientes. Para cumplir con la regla de que solo se puede elegir un único prefijo por cada serie (y no acumular capítulos sueltos de un mismo anime), se implementó un vector booleano de control. Cuando el algoritmo selecciona una opción de prefijo para un anime específico, ese anime se marca como ocupado y se ignoran todas sus demás opciones durante el resto del recorrido lineal.

Criterios Locales de Selección: Se implementaron dos variantes con enfoques distintos para ordenar y elegir las opciones de la lista:

- **Greedy 1 (Costo Combinado):** Este criterio busca aprovechar al máximo ambos recursos disponibles. Ordena todos los prefijos de mayor a menor según el ratio de la satisfacción total (incluyendo el bono si se completa la serie) dividido por la suma del tiempo y la energía que consume el prefijo: $\text{Ratio} = \frac{V_{\text{total}}}{T_{\text{total}} + E_{\text{total}}} [3]$.
- **Greedy 2 (Rentabilidad Temporal):** Diseñado pensando en que el tiempo de transmisión es el factor más crítico. El criterio ordena las opciones usando únicamente los minutos invertidos en el denominador: $\text{Ratio} = \frac{V_{\text{total}}}{T_{\text{total}}}$. El costo de energía se ignora por completo en la ordenación y solo se usa como un filtro al momento de intentar meter el prefijo en la mochila.

Complejidad Temporal Teórica: El algoritmo Greedy ejecuta tres pasos seguidos. Primero, genera todas las opciones posibles de prefijos para los N animes, lo que toma un tiempo proporcional al total de capítulos globales ($K = \sum q_i$). El paso más pesado y determinante es ordenar esta lista completa, lo que se hace con el ordenamiento estándar de C++ (*Introsort*) con una complejidad de $O(K \log K)$. Finalmente, se hace un barrido lineal sobre la lista ordenada que toma tiempo $O(K)$. Por lo tanto, la complejidad temporal total queda acotada por la etapa de ordenación:

$$T(N) = O(K \log K) \quad (4)$$

Como el enunciado limita el total de capítulos globales a un máximo estricto de 700 ($\sum q_i \leq 700$), el valor de K es muy pequeño, explicando por qué los tiempos medidos en tus gráficas son prácticamente instantáneos.

Complejidad Espacial Teórica: La memoria utilizada se limita a la estructura lineal (vector) donde se guardan todos los prefijos generados antes de ordenarlos. Como solo se almacenan datos numéricos planos por cada opción, la complejidad espacial es lineal respecto al total de capítulos:

$$S(N) = O(K) \quad (5)$$

Esto justifica el bajísimo consumo de RAM que se observó en los experimentos para ambas heurísticas.

1.3.3. Programación Dinámica

Para encontrar la solución exacta de forma eficiente en instancias grandes, se adaptó el problema clásico de la mochila bidimensional (2D Knapsack) para manejar grupos de opciones mutuamente excluyentes (*Multiple-Choice Knapsack Problem*) [2]. El estado del problema se representa mediante una matriz $dp[m][e]$, la cual guarda la máxima satisfacción alcanzable usando exactamente m minutos y e unidades de energía.

Para asegurar que el algoritmo elija a lo sumo un solo prefijo por cada anime (evitando sumar capítulos de la misma serie en un mismo paso), se utilizó una técnica de doble matriz. En cada iteración de un anime i , se trabaja sobre una matriz temporal llamada $next_dp$ que inicia como una copia de dp . Los nuevos estados se calculan leyendo estrictamente los valores consolidados del anime anterior almacenados en dp , y los resultados válidos se guardan en $next_dp$. Al terminar de revisar todos los prefijos del anime actual, se actualiza la matriz base haciendo $dp = next_dp$. La relación de recurrencia para un prefijo con costos t_k , c_k y valor v_k es:

$$next_dp[m][e] = \max(next_dp[m][e], dp[m - t_k][e - c_k] + v_k) \quad (6)$$

para todo par (m, e) válido donde $t_k \leq m \leq M$ y $c_k \leq e \leq E$.

Complejidad Temporal Teórica: El algoritmo ejecuta ciclos anidados: el primero recorre los N animes, el segundo pasa por los q_i prefijos de la serie actual, y los ciclos internos recorren la matriz de capacidades desde los límites máximos (M y E) hacia abajo. La complejidad temporal total es:

$$T(N, M, E) = O\left(\sum_{i=1}^N q_i \cdot M \cdot E\right) = O(K \cdot M \cdot E) \quad (7)$$

Esta complejidad es de naturaleza *pseudo-polinomial*, lo que significa que el tiempo de ejecución depende directamente del tamaño de la mochila ($M \times E$). Esto explica por qué en tus experimentos el tiempo varía según el archivo y no aumenta estrictamente de forma lineal con N .

Complejidad Espacial Teórica: Al usar la optimización de doble matriz (dp y next_dp), no se necesita una tercera dimensión para almacenar los estados de cada anime por separado. El programa solo mantiene dos matrices bidimensionales de tamaño $(M+1) \times (E+1)$ en memoria al mismo tiempo. Por lo tanto, la complejidad espacial es totalmente independiente de la cantidad de animes N :

$$S(M, E) = O(M \times E) \quad (8)$$

Esto demuestra teóricamente por qué el uso de memoria en tus experimentos dio exactamente una línea recta horizontal fija en los 21.6 MB para todos los casos de prueba.

2. Resultados y Análisis

Para asegurar la reproducibilidad del análisis, todos los gráficos se generaron de forma automática con el script `plot_generator.py`. Este script procesa directamente los datos crudos obtenidos por los algoritmos en C++ y guarda los archivos resultantes en la carpeta `code/implementation/data/plots/`.

2.1. Análisis de Tiempo de Ejecución

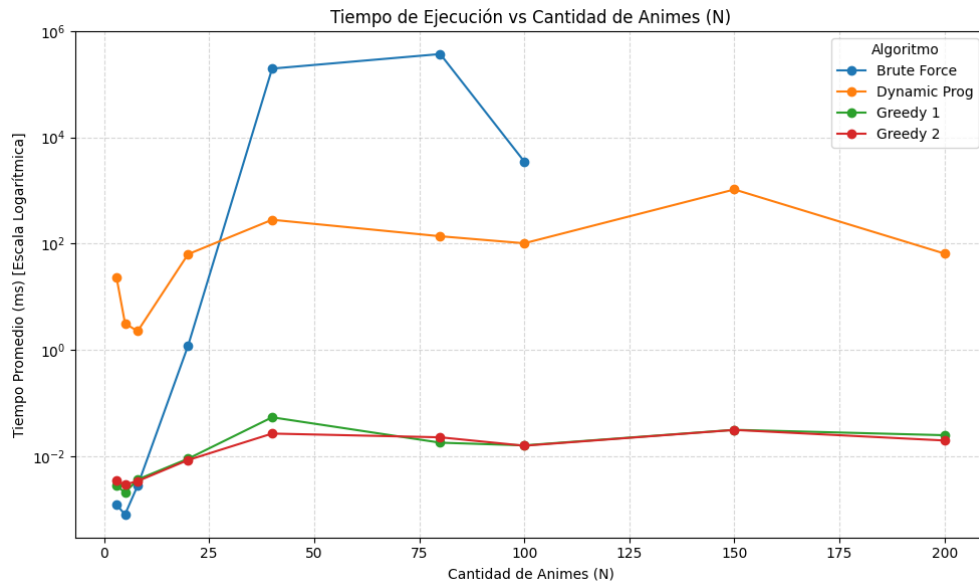


Figura 1: Tiempo de Ejecución promedio (en milisegundos) vs. Cantidad de Animes (N). Eje Y en escala logarítmica.

En la Fig. 1 se puede ver el contraste tan marcado en los tiempos de ejecución de los algoritmos. Lo más relevante es lo que pasa con el algoritmo de Fuerza Bruta (línea azul), la ejecución se tuvo que cortar manualmente después de los casos con $N = 100$. Para las instancias más grandes ($N \geq 150$), el tiempo para evaluar todo el árbol de decisiones creció tanto que superó la capacidad de procesamiento razonable del sistema, lo que causó que se cayera la terminal.

Este corte en la gráfica no es considerable como un error en el script, sino la demostración en la práctica de su complejidad exponencial $O(31^N)$, lo que deja claro que este algoritmo es inviable para problemas grandes.

Por otro lado, la Programación Dinámica (línea naranja) completó todos los casos hasta $N = 200$ en tiempos estables y polinomiales, mostrando variaciones que dependen más de los límites de recursos (M y E) de cada archivo que del tamaño N en sí. Finalmente, ambos algoritmos Greedy corren de forma lineal, con tiempos fraccionarios casi imperceptibles en la base del gráfico.

2.2. Análisis de Uso de Memoria

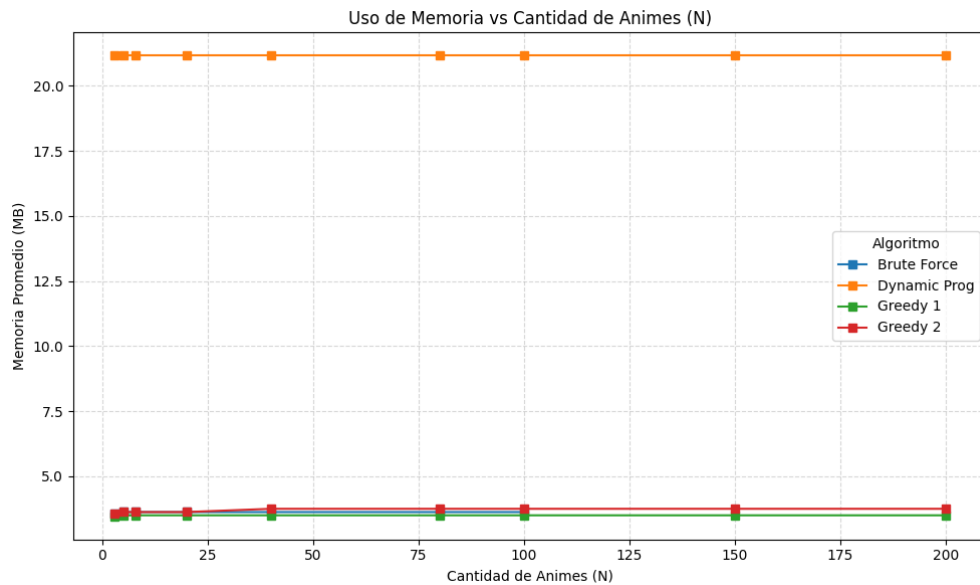


Figura 2: Consumo máximo de Memoria RAM (en MB) vs. Cantidad de Animes (N).

La Fig. 2 muestra el uso de memoria que exige buscar una solución exacta. El algoritmo de Programación Dinámica mantiene un consumo de memoria plano y constante de 21,6 MB, sin importar el número de animes N . Esto confirma exactamente lo esperado por la teoría: el espacio ocupado depende de las dimensiones fijas de la matriz de estados ($M = 3000$ y $E = 500$), dando una complejidad espacial de $O(M \times E)$.

En cambio, Fuerza Bruta y los Greedy muestran un uso de memoria bajísimo y constante (cercano a los 3,5 MB), ya que solo necesitan almacenar las estructuras básicas del problema y la pila de llamadas de la recursión.

2.3. Calidad de la Solución (Heurísticas vs. Óptimo)

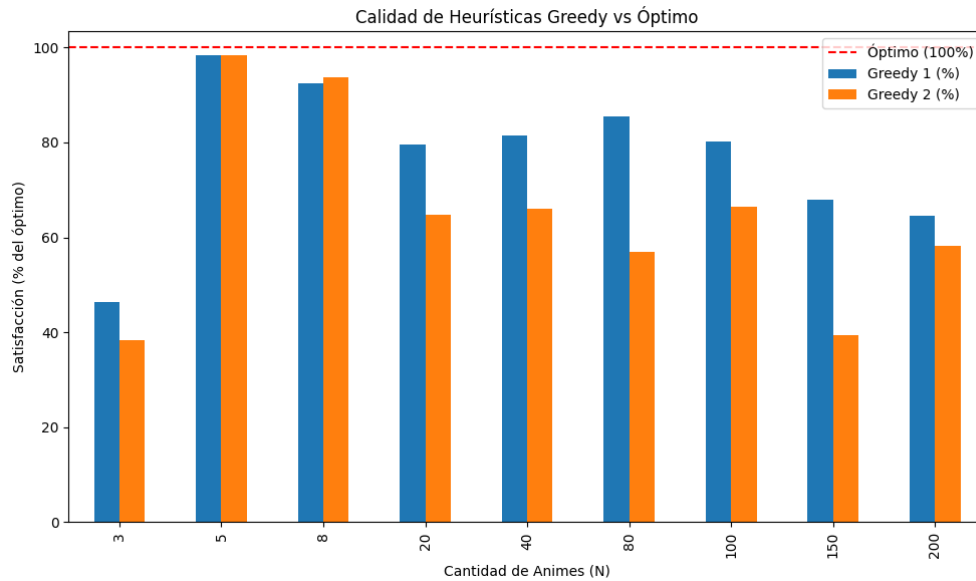


Figura 3: Porcentaje de Satisfacción obtenido por las heurísticas Greedy en relación a la solución óptima

Como los algoritmos Greedy son muy rápidos pero no aseguran encontrar la solución óptima, la Fig. 3 mide qué tan buenos son sus resultados comparados con el óptimo real que entrega la Programación Dinámica.

Se puede observar claramente que **Greedy 1** (que evalúa el ratio usando el costo combinado de tiempo y energía) supera por mucho a **Greedy 2** (que ignora la energía en el ordenamiento). Esto demuestra que, en problemas con restricciones múltiples, armar un criterio local que considere todas las variables del problema da como resultado soluciones subóptimas de mucha mejor calidad.

3. Conclusiones

Los experimentos de esta tarea permitieron comprobar en la práctica cómo escalan los diferentes enfoques algorítmicos frente al problema “AniMaraton”. Quedó en evidencia que Fuerza Bruta es inviable para instancias grandes, lo que se demostró al tener que detener la ejecución manualmente en $N = 100$ debido a los tiempos excesivos provocados por su naturaleza exponencial. Por otro lado, la Programación Dinámica demostró ser la mejor estrategia para asegurar la solución óptima global en tiempos estables, pagando un costo fijo e invariable de memoria de 21.6 MB dictado por su matriz de estados ($O(M \times E)$). Finalmente, las heurísticas Greedy validaron su utilidad como una alternativa extremadamente rápida y ligera si se tolera perder la optimalidad, destacando el rendimiento superior de Greedy 1 sobre Greedy 2 al incluir ambas restricciones (minutos y energía) dentro de su criterio local de selección.

Referencias

- [1] GeeksforGeeks. *0/1 Knapsack Problem using Backtracking*. <https://www.geeksforgeeks.org/0-1-knapsack-problem-dp-10/>. 2024.
- [2] GeeksforGeeks. *2D Knapsack Problem / Multiple Constraints Knapsack*. <https://www.geeksforgeeks.org/2d-knapsack-problem/>. 2024.
- [3] GeeksforGeeks. *Fractional Knapsack Problem: Greedy Approach*. <https://www.geeksforgeeks.org/fractional-knapsack-problem/>. 2024.